Final Evaluation Event - *CyanPeekingMouse*

Liun Grichting (*liunbenjamin.grichting@uzh.ch*)
Lazaro Nicolas Hofmann (*lazaronicolas.hofmann@uzh.ch*)

February 20, 2026

1 Introduction

Our agent CyanPeekingMouse is able to handle a number of user requests: It manages to answer factual & embedding questions, return answers to pure SPARQL queries and recommend movies. With the newest addition it is also able to deal with multimedia questions. For factual questions we directly query the knowledge graph after translating the natural language question into SPARQL. Of course, SPARQL queries are also directly resolved using the KG. In order to recommend movies to a user, we use cosine-similarity and a number of filters to return matching movies, which the user could enjoy. Finally, multimedia questions are resolved by classifying the type of media requested (such as profile images, posters, or backdrops), linking the referenced entity to the closest matching entry in our multimedia index, and returning one of the associated image identifiers. This extension expands CyanPeekingMouse's capabilities beyond purely textual or knowledge-graph-based interactions. Note that we decided to only consider the embedding approach if we fail to retrieve a factual answer, because we want the bot to respond accurately and only give imprecise responses if there is nothing else to return.

2 Capabilities

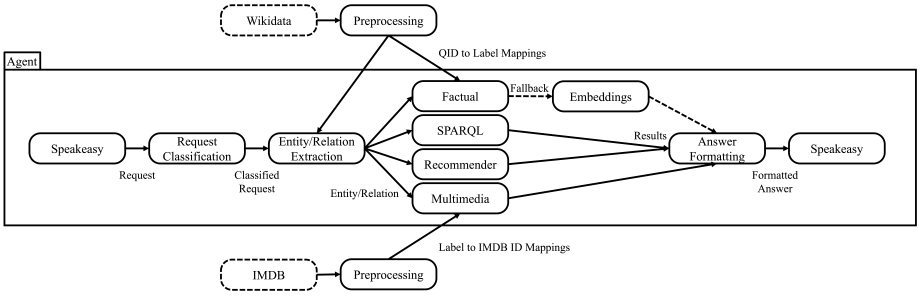

Figure 1: Flow diagram of the agent's processing pipeline

2.1 Factual & Embedding Questions

Our agent uses five main steps to answer factual questions:

1. CyanPeekingMouse listens to the Speakeasy chatroom and waits for a question to be posed.

2. Once the natural language question is received from Speakeasy, extraction.py is used to identify and extract the relevant entities and relations mentioned.

3. Then, using the mappings previously created by label_retrieval.py looks up triples from the knowledge graph. Subsequently, the corresponding URIs are returned such that we can later formulate an appropriate SPARQL query.

4. The actual query is created and executed in factual.py using the RDF graph. In addition, the result is formatted in natural language.

5. Finally, agent.py returns the answer to Speakeasy. Since some relations are missing from the graph not all questions can be answered with the factual approach. In this case the agent falls back to the embedding approach.

6. The embeddings module loads pretrained TransE embeddings for all entities and relations in the graph, as well as the associated identifier and label mappings.

7. The extracted entity–relation pair is interpreted and the missing head or tail entity is attempted to be deduced.

8. The agent computes either *head + relation* (to predict a tail) or *tail - relation* (to predict a head), and then identifies the most likely matching entity in the embedding space.

9. The closest predicted entity is returned to the user together with its label through Speakeasy.

2.2 Recommendations

To make a recommendations based on given titles CyanPeekingMouse follows these steps:

1. The agent extracts all mentioned movies.
2. Then, it connects the links movies to the MovieLens dataset record by string comparison.
3. For each movie provided the bot computes the aggregated cosine-similarity score with all other movies. This means movies that match with multiple titles provided by the user receive higher scores. Of course, the provided titles themselves are removed.
4. Next, we sort the matches in descending order and only keep the top strongest matches.
5. Further we implemented a filter, which removes recommendations that do not fit the genres and production years of the given titles.
6. Finally, the five most fitting recommendations are returned to Speakeasy.

2.3 Multimedia

To handle multimedia-related questions, CyanPeekingMouse follows a pipeline that identifies the requested media type, links the user's query to the correct entity, and retrieves the most appropriate media. The procedure consists of the following steps:

1. The agent extracts the entity mentioned in the question, such as a person or a movie title.
2. It then classifies the type of multimedia requested by scanning the question for keywords. Depending on the phrasing, the agent distinguishes between profile images, posters, and backdrops.
3. Based on the classification, the agent selects the corresponding multimedia index, which is constructed from IMDb data and maps entity names to lists of associated image identifiers.
4. Using a fuzzy matching, the agent links the extracted entity label to the closest matching entry in the selected index. Hence it is able to handle minor spelling mistakes or name variations.
5. If matching images are found, one of the available identifiers is selected at random, such that the media varies between similar queries.

6. Finally, the selected image identifier is returned to Speakeasy in a standardized format.

3 Adopted Methods

In our implementation we used the following libraries:

- *sklearn_crfsuite* [8] is used to train and apply a conditional random field (CRF) for named entity recognition (NER) i.e. it identifies entities from natural language queries.
- *sklearn.metrics.pairwise_distances* [9] is used to compute cosine or euclidean distances between embedding vectors.
- *sklearn.model_selection.train_test_split* [12] splits a NER dataset into training and test sets.
- *joblib* [14] helps us to save and load the trained CRF model so we do not need to retrain it on every run.
- *editdistance* [7] is used to find the closest label match by computing string similarity between the extracted entities or relations to their respective label.
- *pandas* [13] loads and processes the NER dataset, groups words into sentences and handles missing values.
- *numpy* [5] simplifies vector operations such as computing distances or similarities.
- *rdflib* [11] loads the knowledge graph and enables SPARQL queries and traversal of RDF triples.
- *re* [10] helps us to find and match patterns in strings.
- *unicodedata* [10] normalizes text to ensure consistent character encoding.
- *collections.defaultdict* [10] stores structured data.
- *surprise* [6] is used for different tasks such as loading the MovieLens dataset, the SVD matrix factorization algorithm and splitting the data into two parts, one to train and one to test.
- *cosine_similarity* [9] is needed to compute the cosine similarities between movies to make recommendations.
- *collections* [3] is used with *defaultdict* to collect image identifiers without needing to check whether a key already existed.
- *tqdm* [2] is a python package which we used to process lists of of images and IDs and show progress feedback during execution.

- *imdb* [1] is needed to query IMDb for movie and person information. This is a fallback method used when no match was found in the lookup table.
- *typing* [4] is used to simplify code and structure.

4 Examples

In this section, regarding each question type reported above, we provide the answering process of one example question.

Example Question: "*Who is the director of Good Will Hunting?*"

General First Step
Our agent first extracts the entity "*Good Will Hunting*" using either CRF or predetermined delimiters. Next, the relation "*director*" is extracted by comparing individual words with a list of labels. Finally, both the entity and relation are linked to their ID by calculating the lowest string distance.

Factual Approach
The entity and relation are used in a SPARQL query, which returns the answer of the question as a QID. Aftwerwards, the label for the QID is retrieved and in the end the answer is posted into the Speakeasy chat.

Example Recommendation: "*Given that I like The Lion King, Pocahontas, and The Beauty and the Beast, can you recommend some movies? *"
The agent identifies that the movies *Lion King, Pocahontas* and *The Beauty and the Beast* were provided. Next, we compute the aggregated cosine similarity for 500 movies and keep the ones with the best score. These are then filtered by how many genres match with genres of the given movies and whether they are in an certain interval around the given production years. In the end five movies with the most genre matches are returned. The answer is *The Hunchback of Notre Dame, Aladdin, Toy Story, First Kid and Roseanna's Grave (For Roseanna)* which correct since the genres and the production years are close to the given ones.

Multimedia query: "*What does Denzel Washington look like?*" The agent first detects that this is a multimedia request and extracts the entity label *Denzel Washington* from the query. It then examines the wording and identifies the keyword "look like", which is associated with the class of profile images. Next, the agent consults the profile-image index, which was created from IMDb datasets and maps names of people to their available profile photographs. Since the user's extracted entity label may contain minor variations or typographical errors, the agent applies a fuzzy matching step using edit distance to link the query to the closest entry in the index. In this case, the spelling matches directly, and the agent retrieves all profile images associated with Denzel Wash-

ington. If multiple photographs are available, the agent selects one of them at random. Finally, the chosen image identifier is sent back to Speakeasy, allowing the system to display a picture of Denzel Washington.

5 Additional Features

Throughout the course we prioritized the simplicity of the code in order to keep it as understandable as possible. Additionally, we focused on the timeliness such that the user has a solid experience when interacting with our agents. We tried to create a flexible recommendation system with adaptable filters that would recommend suitable but unique movies and and give different results by modifying the filters. Finally, to answer multimedia questions we wanted to ensure that CyanPeekingMouse is able to respond to any question and hence we created a fallback method in case names and titles are missing in the precomputed files. This means in case no information can be retrieved our agent is able to choose an alternative but slower route to retrieve information using the IMDbPY API.

6 Conclusions

Throughout the course we managed to implement the approaches expected. We started off by directly querying the graph using SPARQL queries. The most challenging part of this first step was to understand and utilize the interaction of our agent with Speakeasy. Once we overcame this hurdle, it was clear to us how we can approach the next step. This involved creating the factual and embedding answers where in the latter we found some disappointment in the inaccuracy and unreliability of providing accurate responses. The factual approach on the other hand was convincing and able to provide exact information we were satisfied with. The recommendation system followed, which in our opinion left a lot of room for different approaches. The one we decided to use gives flexible and unique movie suggestions. Finally, we extended our agent with the ability to handle multimedia questions. This required a different strategy than the previous components, since the agent needed to identify when a user was requesting visual information. Overall, the project allowed us to explore a wide range of techniques, from knowledge graph querying to vector embeddings, recommendation filtering, and multimedia retrieval, ultimately resulting in a versatile and well-rounded agent.
Regarding contributions, we decided to divide the work roughly into implementing the agent and writing the report plus attending the evaluations. Therefore, Lazaro took care of realizing the technical details by coding the agent while Liun focused on writing the reports and testing the bot.

References

[1] D. Alberani et al. Imdbpy / cinemagoer python package, 2025. Accessed: December 2025.

[2] F. Bar and contributors. tqdm: Fast, extensible progress bar for python, 2025. Accessed: December 2025.

[3] P. S. Foundation. collections — container datatypes, 2025. Accessed: December 2025.

[4] P. S. Foundation. typing — support for type hints, 2025. Accessed: December 2025.

[5] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.

[6] N. Hug. Surprise: A python library for recommender systems. `http://surpriselib.com`, 2020. Version 1.1.1.

[7] M. John and contributors. editdistance: A fast levenshtein distance computation library for python. `https://pypi.org/project/editdistance/`, 2018. Python package, version 0.5.3.

[8] M. Korobov. sklearn-crfsuite: scikit-learn inspired api for crfsuite. `https://pypi.org/project/sklearn-crfsuite/`, 2015. Version 0.3.6 or latest.

[9] F. Pedregosa, G. Varoquaux, V. Gramfort, M. Wegelin, Y. Nickisch, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. P. McKinney, N. J. Smith, O. Grisel, C. Holdgraf, B. L. Thompson, M. Waskom, and Édouard Duchesnay. Scikit-learn: Machine learning in python, 2011.

[10] Python Core Team. *Python: A dynamic, open source programming language*. Python Software Foundation, 2025. Python version 3.13.5.

[11] RDFLib team. RDFLib: A python library for working with RDF. `https://github.com/RDFLib/rdflib`, 2025. Version 7.2.1.

[12] scikit-learn developers. train_test_split — scikit-learn 1.7.2 documentation, 2025.

[13] The pandas development team. pandas-dev/pandas: Pandas, Feb. 2020.

[14] G. Varoquaux and N. P. Rougier. Joblib: Lightweight pipelining: using Python functions as pipeline jobs. In *Python in Science Conference*, 2018.